

Title

A System of Multiple AI Agents for Automating PETSc Simulation of Plasma in a Tokamak (for Nuclear Fusion Energy).

Presenters

- Sarthak Sharma ss694@buffalo.edu, PhD candidate in Computational and Data Sciences, State University of New York at Buffalo, 0009-0009-6746-169X
- Dr Junchao Zhang jczhang@anl.gov, Division of Mathematics and Computer Science, Argonne National Laboratory, 0000-0003-0367-2358

Keywords

research software engineering; AI agents for scientific computing; PETSc; partial differential equations; magnetically confined plasma; nuclear fusion; shaped tokamak equilibria; safety factor; verification and validation

Abstract

Building a correct, performant HPC simulation from a scientific idea still demands scarce, largely implicit expertise in numerical methods and library APIs. We ask whether a **hierarchical multi-agent AI system** can automate that path for a real fusion-energy problem, and whether the result can be **verified** rather than merely plausible.

We use the PETSc multi-agent system (an open set of Model Context Protocol “AI agents”: https://gitlab.com/petsc/petsc_mcp_servers) to generate, from a plain-language description, a PETSc simulation of the **tokamak Grad–Shafranov equilibrium**: the axisymmetric ideal-magnetohydrodynamics force balance
$$\frac{\partial^2 \psi}{\partial r^2} - \frac{1}{r} \frac{\partial \psi}{\partial r} + \frac{\partial^2 \psi}{\partial z^2} = -\mu_0 r^2 \frac{dp}{d\psi} - \frac{1}{2} \frac{dF^2}{d\psi}$$
 that sets the shape of the magnetically confined plasma. The system mirrors the three-layer architecture of the DoE proposal *Automated Problem-to-Solution Generation for PDE-Based Simulation Science*: a **Mathematical Modeling** agent, a **Numerical Analysis** agent, and an **HPC Code Generation** agent that writes, compiles, and runs PETSc C on the target machine, all driven against Argonne’s Argo LLM gateway (Claude Opus 4.8). A project-owned orchestration driver records every structured intermediate artifact (model, discretization decision, generated source, build / run logs) with full provenance, making runs reproducible and resumable.

From the prompt “*the Grad–Shafranov equilibrium for the magnetically confined plasma in a tokamak,*” the Modeling agent identified the equation and returned its strong and weak forms; the Numerical Analysis agent selected a nonlinear solve (SNES) on a structured grid; and the Code Generation agent produced a **267-line PETSc program** that discretizes the Grad–Shafranov operator on a DMDA with a true Jacobian and solves it with SNES, including a built-in **method-of-manufactured-solutions** check. The generated code compiled and ran on **1 and 4 MPI ranks** on an ANL CELS compute node with **no human edits**. Verification confirms the numerics: the max-norm error is 2.1×10^{-4} on a 65×65 grid and falls to 1.3×10^{-5} on 257×257 , giving an **observed order of accuracy $p = 2.00$** (textbook second order for central differences) under grid refinement, with SNES reporting CONVERGED_FNORM_RELATIVE.

We then drove the same pipeline into a **shaped, real-machine** regime. Using the analytic **Cerfon–Freidberg “one size fits all” Solov’ev** solution as an exact benchmark, a single agent-generated, parameterized PETSc solver reproduces **three physically shaped equilibria**: an **ITER-like D-shape**, a **spherical-tokamak (NSTX-like)** plasma, and a **diverted double-null with magnetic X-points**, set by the inverse aspect ratio, elongation, and triangularity. Each case is **verified at second order ($p = 2.00$)** against the analytic solution and runs on 1 and 4 MPI ranks with no human edits. We compute the **safety-factor profile $q(\psi)$** of each verified equilibrium and cross-check it against the independent **FreeGS** equilibrium code, obtaining agreement to **better than 0.2 %** on identical fields, with the recovered elongation and triangularity matching the targets to around 1 to 2 %. This shows the workflow scales from a toy verification anchor to trustworthy, real-machine-shaped equilibria without changing the generated solver’s structure.

We report **decision-gate metrics**: correctness (identified model, compiled / ran, convergence order), efficiency (wall-clock and LLM / tool calls), and human effort (0 lines of solver code hand-written), and we contribute portability and robustness fixes back to the multi-agent system (CWD-independent server resolution; graceful operation where the documentation / RAG services are unavailable; a code-generation loop that captures results reliably). The work is a concrete demonstration that verification-driven, multi-agent problem-to-solution generation can produce trustworthy HPC simulation code for a flagship fusion-energy problem.

References

1. B. Smith, H. Zhang, J. Zhang, S. Balay, L. Chen, M. Keçeli, L. C. McInnes. *Improving usability and productivity of PETSc with agent-based workflows*. 2026. doi:10.13140/RG.2.2.35234.80326.
2. L. C. McInnes et al. *Automated Problem-to-Solution Generation for PDE-Based Simulation Science*. Argonne National Laboratory proposal, DE-FOA-0003612, 2026.
3. S. Balay et al. *PETSc/TAO Users Manual*. ANL-21/39 Rev. 3.25, Argonne National Laboratory, 2026. <https://petsc.org>
4. PETSc multi-agent MCP servers. https://gitlab.com/petsc/petsc_mcp_servers; <https://mcp.petsc-ai.org>
5. H. Grad, H. Rubin. *Hydromagnetic equilibria and force-free fields*. Proceedings of the Second UN Conference on the Peaceful Uses of Atomic Energy, 1958. (Grad–Shafranov equation.)
6. A. J. Cerfon, J. P. Freidberg. “One size fits all” analytic solutions to the Grad–Shafranov equation. *Physics of Plasmas* 17, 032502, 2010.
7. This work: https://github.com/engineer-scientist/petsc_mcp_servers_tokamak

Connection to Mission, Goals, and Interests of US-RSE Community

Research software engineers increasingly sit between AI code-generation tools and the trusted, performance-critical libraries that underpin computational science. This submission speaks directly to that intersection. It is a candid RSE case study of *using* an agentic AI system on a real HPC library (PETSc) for a real DoE-mission problem (fusion-energy plasma), rather than a toy benchmark, including the unglamorous engineering that makes such systems actually work on a shared compute node: environment and dependency wrangling, making services degrade gracefully when optional components are absent, working-directory-independent process spawning, and reproducible provenance-tracked runs. We treat **verification as a first-class deliverable**: the generated simulation is checked against an exact solution and a grid-convergence study, reflecting the RSE community’s emphasis on correctness, testing, and reproducibility over raw output. Finally, our improvements are contributed **back to the open-source multi-agent project** as patches, embodying the community’s values of open, sustainable, and collaborative software. We hope the poster sparks discussion about how RSEs can responsibly adopt, evaluate, and harden AI-assisted workflows for scientific software.